Market and Reference Data: A Specification for the Ledger Platform

MVP, v1.0

Data Engineering Specification

May 2026

1 Audience and the Three Claims This Document Defends

This document is for the data engineering team building the data substrate consumed by the Ledger platform. The reader is competent at SQL, Avro/Protobuf, ETL, and warehouse design; the reader is not assumed to be a quant or a derivatives specialist. Every term used here is defined on first use.

The Ledger is a system that records portfolio positions, transactions, and valuations. It depends on a substrate of data — product definitions, market quotes, identifiers, calendars, events from outside parties — which must be ingested, stored, and queried in a disciplined way. This document specifies that substrate.

Three claims are load-bearing throughout. Everything else in this document either justifies one of them or specifies how it is implemented.

Claim 1 (CDM as the wire vocabulary). The platform's wire formats and reference identifiers conform to the FINOS Common Domain Model (CDM), version 6.0.0, wherever CDM defines a type that fits. The reason is concrete: if two systems exchange a trade or an instrument identifier and disagree on the field structure, every downstream check (reconciliation, valuation, regulatory mapping) starts from inconsistent inputs and produces garbage. CDM gives every party the same field names with the same meanings.

Claim 2 (every datum is attested; nothing is overwritten). Every record entering the system carries an attestation envelope: who said it, when they said it (*observation time*), when we learned it (*known time*), and a cryptographic signature. No record is ever overwritten in place. The reason is concrete: an auditor who asks "reproduce the end-of-day NAV you published on 14 March" must be able to retrieve the price you used, the party who attested it, and the moment your system received it. If any of those four facts has been silently overwritten by a later correction, the auditor cannot reproduce the NAV and the platform's audit story is finished.

Claim 3 (bitemporal storage; time-travel queries are first-class). Every record on the boundary carries two timestamps: *observation time* (when the underlying event happened in the world) and *known time* (when our system first admitted the record). When a vendor restates a price for an earlier observation time, that restatement is a new row, not an update. The reason is concrete: vendors restate. A FX print stamped 15:30 on 27 April may be corrected on 30 April. A query asking "what did we believe at 09:00 on 28 April?" must return the original print, not the correction. A query asking "what is our best estimate today of the truth at 15:30 on 27 April?" must return the correction. Single-axis "as-of" storage cannot answer both questions; bitemporal storage answers both by construction.

2 The Platform in Half a Page

The Ledger records portfolio activity as transactions of *moves* between *wallets*. A wallet holds quantities of *units* (a unit is any traded thing: a currency, a share, a bond, an OTC derivative, a

smart-contract obligation). Every transaction conserves quantity per unit: what leaves one wallet enters another. The platform is not a pricing engine; portfolio value is computed by combining wallet balances with prices supplied by the data substrate this document specifies.

The data engineering team owns the boundary of that platform. It ingests records from outside parties — market data vendors, GLEIF, calculation agents, custodians, exchanges, regulators — attaches an attestation envelope to each, stores them bitemporally, and exposes typed queries to the rest of the platform. Everything inside the Ledger (move stream, position state, valuation records) is downstream of this boundary and is out of scope here.

3 CDM 6.0.0 as the Wire Vocabulary

CDM is the FINOS Common Domain Model, expressed in the Rosetta DSL. It defines named types for products, parties, events, and reference data, with stable Rosetta source files in `rosetta-source/src/main/rosetta/`.

We adopt CDM as the wire format under three rules:

Rule 1 (CDM-direct types are wire and storage). For every concept this document marks as Direct against CDM 6.0.0, the CDM type is the wire format on ingest and the storage shape in the warehouse. No vendor-specific column names. Without this rule, every vendor change becomes a schema migration, and a team six months from now reading a stored record cannot tell what it means without the vendor's documentation.

Rule 2 (CDM-partial types are wire; storage extends with named local fields). Where CDM covers identity but not the operational fields we need (e.g. board lot, voting rights, dividend policy reference), the CDM type is the wire format and storage adds named columns. Local extensions are documented in the data dictionary; without that documentation, the difference between "CDM said so" and "we added this" is lost on day one.

Rule 3 (Missing types are local now, with a CDM extension proposal logged). Where CDM has no type (calibrated curves, sanctions lists, withholding tables, clock authority), the storage shape is local and a CDM extension proposal is logged in the data dictionary. Without the logged proposal, the team in eighteen months cannot tell which gaps the platform took on intentionally and which are accidents.

The Rosetta source files cited in Table 1 — `base-datetime-type.rosetta`, `base-datetime-enum.rosetta`, `base-datetime-daycount-enum.rosetta`, `base-staticdata-asset-common-type.rosetta`, `base-staticdata-party-type.rosetta`, `legaldocumentation-common-type.rosetta`, `legaldocumentation-common-enum.rosetta`, `observable-asset-type.rosetta`, `observable-event-type.rosetta`, `event-common-type.rosetta`, `product-template-type.rosetta` — live under `rosetta-source/src/main/rosetta/` on `finos/common-domain-model` and were verified live on 2026-05-10. We pin to specific commit SHAs, not branch heads, because branch heads move and a wire format that depends on whatever was on `master` on a given Tuesday is not a wire format. The pin is reviewed quarterly; a pin bump is a normal change-control ticket.

4 The Attestation Envelope

Every record entering the system carries an *attestation envelope*. The envelope is one named field-set carried on every payload, not a separate table.[1]

Definition 1 (Attestation Envelope). *The attestation envelope is the tuple*

$$(attestor\_lei,\ signature,\ payload\_hash,\ source\_message\_id,\ gateway,\ gateway\_time,\ idempotency\_token,\ t_{obs},\ t_{known}).$$

Field	Type	Constraint
`attestor_lei`	string(20)	ISO 17442 LEI; resolves in trust registry.
`signature`	bytes (per scheme)	Covers the canonical payload bytes plus the envelope minus this field; see Rule 5.
`payload_hash`	bytes(32)	SHA-256 of the canonicalised payload. Binds envelope to payload.
`source_message_id`	string ≤ 256	Vendor-supplied identifier; opaque.
`gateway`	enum (pinned closed list)	Identifies the ingress gateway, e.g. `vendor-a-fix-gw`.
`gateway_time`	int64 nanos UTC	Wall clock at gateway commit.
`idempotency_token`	bytes(16)	UUIDv7. Replays of the same token produce no second record.
t_{obs}, t_{known}	int64 nanos UTC	Bitemporal axes (§5).

Rule 4 (No record without an envelope). Every record on every in-scope ingest table carries the full envelope. Records missing any field are rejected at the gateway and the rejection itself is recorded as a typed event carrying the inbound envelope and the failing field. Without this, a record with an unknown attestor or no signature can be quietly mixed with attested records, and the audit chain is broken at the boundary; without the typed rejection event, the auditor asking "what did you reject yesterday and why?" has nothing to read.

Rule 5 (Signatures verified at the gateway, retained for replay). The gateway verifies the signature against the attestor's public key, held in the trust registry (§6). The signature covers the canonicalised payload bytes (under the canonicalisation rule named in the consumed VersionPin) concatenated with the envelope-minus-signature, itself canonicalised under the same VersionPin rule (RFC 8785 JCS for JSON wire; Avro / Protobuf canonical-form rules for those wires); the verification therefore binds payload, attestor, and bitemporal axes together under one canonicalisation pin. Verified signatures are stored with the record. Public verification keys are append-only: rotating a key adds a new key, retains the old one, and marks the old one with a `key_valid_to` timestamp; revoked keys remain readable for replay verification but are not eligible to admit new records (see Trust Registry, §6). Without retention, replaying an old record to reproduce a historical NAV fails because the key that signed it is no longer available; without binding the signature to the canonical payload, a vendor (or a man-in-the-middle) can swap the payload while keeping a valid envelope and the gateway will not notice.

Rule 6 (Idempotency token deduplicates replays). Every inbound payload carries an idempotency token; the gateway maintains a deduplication table keyed by token. Re-presenting a token returns the original outcome rather than producing a second record. Without this, a vendor retry — because their network blipped — creates two records for the same observation, and downstream aggregations double-count.

[1] The parent data spec (L_9 envelope decomposition) splits the envelope into an idempotency-token carrier and a signed-envelope carrier; for this document, one field-set.

5 Bitemporal Storage and Time-Travel Queries

Definition 2 (Observation time and known time). *The* observation time t_{obs} *is the wall-clock instant the underlying event in the world occurred (the moment a price printed, the moment a corporate-action effective date applied). The* known time t_{known} *is the wall-clock instant our system first admitted the record. By default* t_{known} = *gateway_time, the envelope field; a divergent* t_{known} *is permitted only when the record was admitted via a controlled offline pathway (e.g. replay from a vendor archive) and the divergence is recorded in the typed admission event. Both timestamps are nanosecond-resolution UTC. The constraint* $t_{obs} \leq t_{known} \leq now()$ *is enforced at the gateway.*

Rule 7 (Restatement is a new row, never an update). When a vendor corrects a record for an earlier t_{obs}, the correction is inserted as a new row with the same t_{obs}, a later t_{known}, and a pointer to the row it restates. The original row is not modified. Without this rule, a query asking what we believed yesterday cannot be answered, because yesterday's belief has been overwritten by today's correction.

Rule 8 (Two query modes, both first-class). The query API exposes two modes:

- `as_of`(t_k): returns the snapshot of the table as it was *known* at t_k. Reflects what the system knew then, not what was true.
- `with_corrections_through`(t_o, t'_k): returns the best estimate, as of knowledge time t'_k, of the truth at observation time t_o.

Neither mode is derivable from the other. Without both, an auditor's question about yesterday's published number and a risk team's question about "what really happened" collapse into a single ambiguous query and the answer depends on what the implementer felt like that day.

Example (Vendor restatement of a FX print). A vendor publishes EUR/USD = 1.0852 with t_{obs} = 2026-04-27 15:30:00.000Z, received at t_{known} = 2026-04-27 15:30:00.123Z. On 2026-04-30 09:00:00Z the vendor restates the same observation time to 1.0853. The table holds two rows:

Row	payload	t_{obs}	t_{known}	restate_link
R_1	1.0852	2026-04-27 15:30:00.000Z	2026-04-27 15:30:00.123Z	—
R_2	1.0853	2026-04-27 15:30:00.000Z	2026-04-30 09:00:00.000Z	R_1

Query 1: `as_of`**(2026-04-29 00:00:00Z).** Returns 1.0852. As of that knowledge time, R_2 did not yet exist; the only row whose $t_{\text{known}} \leq$ 2026-04-29 was R_1. This is the answer the auditor needs to reproduce the NAV the firm published on 28 April.

Query 2: `with_corrections_through`**(2026-04-27 15:30:00.000Z, 2026-04-30 10:00:00Z).** Returns 1.0853. Among rows with matching t_{obs}, take the row with the maximum $t_{\text{known}} \leq$ 2026-04-30 10:00:00Z; that row is R_2. This is the answer the risk team needs when restating yesterday's exposure with current best knowledge.

Rule 9 (Bitemporal indexing is mandatory on definitions and observations). The two-axis index is mandatory on every table that holds either definitional records (product terms, identifiers, calendars, legal agreements, clock authority) or observational records (raw market prints, calibrated curves, lifecycle events from outside parties, external confirmations). Single-axis "as-of" is forbidden on these tables. Reference tables that the platform itself owns and never restates (internal policy configuration) need not be bitemporal, but if there is any chance the source can correct itself, two axes are mandatory.

Rule 10 (Primary key, uniqueness, and tie-break). The primary key on every bitemporal table is (business_key, t_{obs}, t_{known}); uniqueness is enforced on (business_key, t_{known}). A covering index on (business_key, t_{obs}, t_{known}) is mandatory; without it, a single `as_of` or `with_corrections_through`

query against a multi-billion-row table degrades into a full scan and the latency budget collapses. On equal t_{known} inside a single (t_{obs}) partition, ties resolve to the row whose admission outcome is *multi-source consensus* ahead of *unique authority*; if neither row qualifies, the query is treated as undefined: the consumer receives a typed undefined-tie event referencing both rows by primary key, and the event is recorded in the break register for operator review. The restatement chain length per business key is bounded by a configured constant (typical: 32); restatements beyond that bound enter the break register and require manual review.

Operating envelope

The substrate is sized to the following envelope. Numbers are order-of-magnitude commitments for the MVP and are revisited in production sizing.

- **Ingest rate:** up to 5×10^7 rows / day across all market-observation tables; 10^6 rows / day across reference tables. Spikes to 10× steady-state during open / close.
- **Retention horizon:** 7 years for tables subject to SOX, MiFIR, EMIR, CFTC Part 49; longer of regulator horizon or (business-key non-zero + 7y) for instruments with no maturity; bitemporal restatements counted toward retention, never pruned within the horizon. Required because the audit ask "reproduce the NAV from 2021-Q4" must succeed in 2028.
- **Online queryability:** records within the retention horizon remain queryable through the same `as_of` / `with_corrections_through` API at the same latency budget; no offline restore step. Without this, "retained" degrades to "buried" and the seven-year audit ask becomes a multi-day project.
- **Hot-path signature verification:** p99 $\leq 500\,\mu$s per record. The gateway maintains an in-process LRU cache of attestor public keys, sized to hold all keys with status *Active*; cache misses fall through to the trust registry with a 5 ms timeout. *Fail-closed*: a timeout, a cache miss that does not resolve, or any registry error rejects the record with a typed envelope-rejection event; fail-open is forbidden. Cache invalidation: any insert into the trust registry's key table publishes a typed signal to all gateway processes; a gateway that has not acknowledged within 60 s is taken out of rotation. Verifier-thread sizing: ∼4 verifier threads per gateway, two gateways minimum for redundancy, sized to absorb the 10× open / close burst at the p99 budget.
- **Trust-registry revocation, dual gate:** from registry insert to last gateway acknowledging refresh, ≤ 60 s. *Additionally*, every snapshot consumed by a downstream computation re-evaluates each input record's signing key against the authoritative registry as of the snapshot's t_{known}; any record whose key is Revoked at that time is quarantined regardless of whether it passed gateway admission. Without this second gate, a key revoked because of compromise still admits records during the 60 s propagation window — the very control the registry is supposed to prevent.
- **Restatement window:** a vendor restatement carrying $t_{\text{known}} - t_{\text{obs}} > 90$ days is admitted but flagged; consumer tooling treats long-tail restatements as advisory until manually reviewed. The 33rd restatement on a single business key (one beyond the chain bound) is not silently dropped: it is admitted into the break register with the prior chain attached, and downstream consumers see a typed event rather than a missing row. Without the bound, a vendor sending a six-year-old restatement triggers a fan-out to every dependent computation in the retention horizon and the platform is denial-of-serviced by its own correctness machinery.
- **Calibrated-curve storage:** calibrated objects are stored as compressed knot-point representations; identical objects are deduplicated by content hash across snapshots; full-grid reconstruction is a query-time concern, not a storage one. Without this, a daily 10 k-instrument vol surface stored in full grid breaks the storage budget within a year.
- **VersionPin storage:** pins are deduplicated by content hash; each unique pin is stored once and referenced by content-addressed id from every snapshot that consumed it.

6 The Data Catalogue

The boundary surface is fifteen concepts. Table 1 gives, for each, a one-sentence definition, which of the three claims it primarily serves, and its CDM 6.0.0 mapping (Direct, Partial, or Missing). Concepts marked Partial or Missing carry a note pointing to the gap. Sub-rows in the same concept (e.g. ReferenceMaster's five sub-tables) share the concept row.

Table 1: The fifteen-concept ingest catalogue.

Concept	Definition	Claim	CDM 6.0.0 mapping
ProductTerms	Versioned OTC contractual terms keyed by unit (strike, maturity, underlier, day-count, payoff). Covers OTC EconomicTerms; the listed-instrument identity branch is in InstrumentMaster.	1, 2, 3	Direct on `NonTransferableProduct` in `product-template-type.rosetta`. Wire and storage.
Instrument-Master	Bitemporal map keyed by (authority, instrument id, version) reconciling vendor identifiers (ISIN, CUSIP, FIGI, exchange code). Covers listed equities, listed derivatives, bonds.	1, 2, 3	Partial via `Security` (a branch of `Instrument`, itself a branch of the `Asset` choice; `Security extends InstrumentBase`). Identity direct via `AssetIdentifier`; operational fields (board lot, voting rights, dividend policy ref) local. Types in `base-staticdata-asset-common-type.rosetta`.
PartyLEI	Bitemporal LEI record with status (Issued, Lapsed, Retired, Annulled, Duplicate); BIC and MIC indexes. Lapse is a new bitemporal row, not a deletion; the status at the record's t_{obs} is what binds.	1, 2, 3	Direct on `Party.partyId` (cardinality `1..*`, type `PartyIdentifier`) with `identifierType = LEI`, conforming to ISO 17442. Type in `base-staticdata-party-type.rosetta`.
Calendar-Convention	Bitemporal calendar payload (business centres, holiday lists) plus day-count and business-day-convention enumerations. The controlling calendar for a trade is the one named in the LegalAgreement or the confirmation, not a generic firm-wide default.	1, 3	Direct on `BusinessCenters` (`base-datetime-type.rosetta`), `BusinessCenterEnum` (`base-datetime-enum.rosetta`), and `DayCountFractionEnum` (`base-datetime-daycount-enum.rosetta`).
LegalAgreement	Bilaterally signed Master / Schedule / CSA / Para-11 stack keyed by agreement id; per-document hashes; later items in the stack override earlier.	1, 2	Direct on `LegalAgreement extends LegalAgreementBase` (`legaldocumentation-common-type.rosetta`) with `LegalAgreementTypeEnum` (`legaldocumentation-common-enum.rosetta`).
RawMarket-Observation	Append-only bitemporal stream of vendor quotes, trade prints, fixings, and FX prints with attestation envelope and admission outcome. The four shapes have different attestor semantics (e.g. a fixing carries a calculation-agent attestor, a trade print carries an exchange attestor) and are typed accordingly.	2, 3	Partial on `Observation` in `observable-event-type.rosetta`. CDM `Observation` is a root type and is independent of lifecycle events; the gap is that it lacks the admission-outcome enumeration {multi-source consensus, unique authority, quarantined} and lacks bitemporal axes. Local extension adds both.
Calibrated-MarketObject	Versioned snapshot of calibrated curves and surfaces (yield, volatility, hazard, FX) with admissibility certificate.	2, 3	Missing — no CDM type for calibrated curves or surfaces. Extension shape: a proposed namespace adds `CalibratedMarketObject` alongside `Observation` with mandatory fields `kind: CalibratedObjectKindEnum (1..1)` (`YieldCurve` | `VolSurface` | `FxSurface` | `HazardCurve`), `valuationDate: date (1..1)`, `inputObservationKey: Observation (1..*)`, `parameterisation` (polymorphic sub-type per `kind`), and `arbFreeCertificate: ArbFreeCertificate (0..1)` recording the admissibility witness.
LifecycleOracle	Append-only bitemporal stream of authority-signed lifecycle events (corporate action, fixing, exercise, default, reset, novation, locate, recall, manufactured payment, etc.).	1, 2, 3	Partial on `BusinessEvent` (event-model root), `Reset` (`event-common-type.rosetta`), and `CorporateAction` (`event-common-type.rosetta`; not `CorporateActionEvent`). Securities-lending events (Locate, Recall, Rehypothecation) and ManufacturedPayment Missing; ISLA CDM working group coordinates extensions.

Concept	Definition	Claim	CDM 6.0.0 mapping
External-Confirmation	Append-only stream of inbound confirmations keyed by (transaction id, external message id). Standardised channels include FpML feeds, ISDA Create, ISDA Notices Hub, and the DRR submission acknowledgements.	2, 3	Partial via dense FpML synonyms in CDM 6.x (`ingest-fpml-confirmation-*-func.rosetta`); ISO 20022 mapping (sese.025, sese.023, camt.053, camt.054) is local with an aspirational synonym proposal logged.
ReferenceMaster	Cross-cutting authoritative tables: sanctions, withholding-tax, CCP-margin, index composition, corporate-action reference schedules.	1, 2, 3	Mixed. Index composition Partial via `choice Index` (`observable-asset-type.rosetta`; branches `CreditIndex`, `EquityIndex`, `InterestRateIndex`, `ForeignExchangeRateIndex`, `OtherIndex`). Corporate-action reference Partial via `CorporateAction`. Sanctions, withholding, CCP-margin Missing.
ClockAuthority	Bitemporal time-source authority record (NTP / PTP / GNSS / atomic) with leap-second policy version and signed offset. Every t_{obs} and t_{known} in every in-scope table references one.	2, 3	Missing. Local.
Trust Registry	Append-only registry of attestors and the authority assumptions they back. Row shape: `(attestor_lei, status enum {Active, Suspended, Revoked}, public_key bytes, key_valid_from int64, key_valid_to int64 NULL, authority_assumption_ref FK NULL)`; PK `(attestor_lei, key_valid_from)`, with the partial-uniqueness constraint that exactly one row per `attestor_lei` has `key_valid_to IS NULL` (the currently-live key). Rotation sets `key_valid_to` on the old row and inserts a new row; revoked keys are retained for replay verification but the admission rule does not treat them as live authority. The `authority_assumption_ref` resolves into a sibling table `AuthorityAssumption(observable_key, attestor_lei, valid_from, valid_to, reason_text, signed_by_owner_lei)`, where `signed_by_owner_lei` is the internal owner who signed off on the single-source dependency.	2	Missing. Keyed on `attestor_lei`, the same value carried in the AttestationEnvelope, which corresponds to `Party.partyId.identifier` with `PartyIdentifierTypeEnum = LEI`.
Attestation-Envelope	Cross-cutting field-set carried on every record (§4). Not a table.	2	Missing. Local; threaded through every CDM-typed payload.
Bitemporal axes	The two timestamps $(t_{\text{obs}}, t_{\text{known}})$ and the two query modes (§5). Not a table.	3	Missing. Local; layered over any CDM type.
VersionPin sidecar	Per-snapshot pin record carrying seven version axes plus the trust-registry version: *component*, *schema*, *contract*, *model*, *refdata*, *regulator-rule-set*, *canonicalisation*, plus `trust_registry_version`. Not a table; carried with each consumed snapshot, deduplicated by content hash.	3	Missing. Local.

7 Cross-Cutting Rules

The rules in §4 and §5 bind every concept in Table 1. Four further rules are cross-cutting and worth stating explicitly because each closes a recurring failure mode.

Rule 11 (Multi-source admission is the default; single-source admission requires a registered assumption). Every record on `RawMarketObservation` carries an admission outcome, taking exactly one value from {*multi-source consensus*, *unique authority*, *quarantined*}. Records labelled *quarantined* do not enter snapshots consumed by downstream calibration. Single-source admission — a record from a sole vendor with no second source — is permitted only when the record carries a named reference to an entry in the trust registry recording an explicit *authority assumption* for that source on that observable. Without the registered assumption, a single vendor's outage or feed corruption silently becomes the platform's truth, and the platform inherits a trust dependency that no one has signed off on.

Rule 12 (LegalAgreement is a stack with a defined override walk). A LegalAgreement is a stack of four document kinds in order: *Master*, *Schedule*, *CSA*, *Para 11*. Each is an independent bitemporal row keyed by `agreement_id` with its own document hash. To resolve an attribute (say, governing law or eligible-collateral set) at a given t_{obs}, walk the stack head-to-tail and take the last non-null override. Without a defined walk, two implementers comparing notes on the same trade reconstruct different governing terms and the dispute resolution depends on whose code ran last.

Rule 13 (Free-text fields are forbidden in invariant-bearing positions). Status fields, event kinds, day-count fractions, business-day conventions, agreement types, admission outcomes, and message kinds are stored as enumerations from a pinned closed list. Free text is permitted only for opaque vendor identifiers and human-readable descriptions, never in fields that any rule depends on. Without this, an unknown enum value silently passes a string-equality check, no alarm fires, and a downstream join over the field returns nothing or returns the wrong rows.

Rule 14 (Snapshots consumed by replay carry a VersionPin). Every snapshot consumed by a downstream computation (a calibration, a valuation, a regulatory submission) is tagged with a VersionPin recording seven version axes — *component* (deployed code SHAs), *schema* (Avro/Protobuf message versions), *contract* (CDM `NonTransferableProduct` / smart-contract version), *model* (calibration model version), *refdata* (effective reference-data version per authority), *regulator-rule-set* (DRR rule-set version per regulator), *canonicalisation* (e.g. RFC 8785 JSON canonicalisation) — plus the `trust_registry_version` in force at consumption time. Without these pins, replaying yesterday's computation against today's schema produces a different bit-pattern, the hash check fails, and the replay claim collapses; without `trust_registry_version`, a key revocation that arrived after the original computation silently changes which inputs would now pass admission.

8 Acceptance Tests for the Data Substrate

The data substrate ships when the following tests pass. Each test is the operational form of one of the rules above, so the tests are a checklist for the rules, not new requirements.

1. **Envelope completeness.** For every in-scope ingest table, every row carries a non-null envelope. Rejection events are recorded as typed events carrying the inbound envelope and the failing field; the rejection rate is published per attestor.
2. **Signature verification.** For a sample of 10^5 historical rows *stratified by attestor*, signatures verify against the trust registry's public keys at the snapshot pinned at t_{known}. Without stratification, a long-tail attestor with a quietly broken key is masked by the volume of a healthy one.
3. **Bitemporal restatement round-trip.** Inject the EUR/USD restatement fixture from §5 (the R_1/R_2 pair) and verify both `as_of`(2026-04-29 00:00:00Z) and `with_corrections_through`(2026-

04-27 15:30:00.000Z, 2026-04-30 10:00:00Z) return the documented values. The fixture and its expected outputs are the canonical regression fixture.

4. **Admission outcome closure.** For every `RawMarketObservation` row, the admission outcome lies in the three-element set, every *unique authority* row resolves to a live trust-registry assumption, and *no row labelled* `quarantined` *appears in any snapshot consumed by a downstream computation.*
5. **CDM mapping round-trip.** For every Direct concept, a stored row round-trips through the CDM Rosetta-defined wire format with bit-identical canonical bytes under the canonicalisation rule named in the consumed VersionPin (RFC 8785 JCS for JSON; the Rosetta canonical form for the Rosetta DSL).
6. **No silent overwrite (continuous, not full-scan).** The hash chain over rows of each business key is maintained on insert; the integrity check verifies the chain head, not a full-table scan. The check confirms that, for every business key, the highest-t_{known} row is not byte-equal to any prior row of that business key.
7. **VersionPin presence.** Every snapshot delivered to a consumer carries a VersionPin and the seven version axes plus `trust_registry_version` resolve. Snapshots without one are rejected at delivery.
8. **Clock authority pin.** Every t_{obs} and t_{known} resolves to a ClockAuthority record live at t_{known}, and the signed offset at that time falls within the authority's published tolerance band.

9 Out of Scope

The following are deliberately out of scope for this document.

- **The move stream, position state, and unit status.** These are the Ledger's internal record of what happened, derived from ingested data. They are written by the platform, not by the data team.
- **Valuation records, pricing methodology, Greeks, the Pricing DAG.** The data team supplies CalibratedMarketObject; the mathematics that consumes it is the valuation team's concern.
- **Obligations, the break register, regulatory submissions.** These are platform-produced artefacts.
- **Storage technology and wire encoding choices.** This document specifies data shape and discipline; whether the warehouse is columnar Parquet, an immutable log, or a relational store is a separate decision, as is the choice between Avro, Protobuf, or CBOR for the wire format. The CDM types pin the field structure regardless.
- **Vendor selection, KYC/AML controls beyond the fields they require, legal-entity onboarding workflows.**
- **Verification methodology beyond the acceptance tests in §8.** Property-based testing, mutation testing, and the test pyramid are specified in the formal corpus, not here.

10 Glossary

Attestation envelope
: The named field-set carried on every record: attestor LEI, signature, payload hash, source message id, gateway, gateway time, idempotency token, t_{obs}, t_{known}.

Attestor
: An external authority (vendor, regulator, calculation agent, custodian) whose statements enter the system only through a signed envelope.

Bitemporal
: Two-axis indexing of records by observation time and known time; both axes are query-able.

CDM
: FINOS Common Domain Model, version 6.0.0, expressed in the Rosetta DSL.

GLEIF
Global Legal Entity Identifier Foundation, the authority that issues LEIs.

Idempotency token
An identifier carried with each inbound payload; replays of the same token produce no second record.

Known time (t_{known})
Wall-clock instant the system first admitted a record.

LEI Legal Entity Identifier, ISO 17442.

Observation time (t_{obs})
Wall-clock instant the underlying event occurred in the world.

Restatement
A correction to a prior record, stored as a new row with the same t_{obs}, a later t_{known}, and a pointer back to the row it restates.

Trust registry
The local table of attestors, their statuses, their public verification keys, and any registered single-source authority assumptions.

Unit
Any traded thing recorded as a balance in a wallet (currency, share, bond, OTC derivative, smart-contract obligation).

VersionPin
The per-snapshot record naming the seven version axes (component, schema, contract, model, refdata, regulator-rule-set, canonicalisation) plus the trust-registry version in force at consumption time.

Wallet
A named account inside the Ledger holding quantities of units. Not a custody account or a bank account.

Document Version: v1.0 **Date:** May 2026 **Classification:** MVP specification, data engineering audience.